

# Spring Core & Maven - Lab Answers

## Exercise 1: Configuring a Basic Spring Application

### 1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.library</groupId>
  <artifactId>LibraryManagement</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <properties>
    <maven.compiler.source>1.8</maven.compiler.source>
    <maven.compiler.target>1.8</maven.compiler.target>
    <spring.version>5.3.30</spring.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${spring.version}</version>
    </dependency>
  </dependencies>
</project>
```

### 2. BookRepository.java

***src/main/java/com/library/repository/BookRepository.java***

```
package com.library.repository;

public class BookRepository {

    public void addBook(String title) {
        System.out.println("Book added to repository: " + title);
    }
}
```

### 3. BookService.java

***src/main/java/com/library/service/BookService.java***

```
package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }
}
```

```

    public void registerBook(String title) {
        System.out.println("Service layer received request for: " + title);
        bookRepository.addBook(title);
    }
}

```

## 4. applicationContext.xml

### *src/main/resources/applicationContext.xml*

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService" />

</beans>

```

## 5. LibraryManagementApplication.java (main class)

```

package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService bookService = (BookService) context.getBean("bookService");
        System.out.println("Context loaded, bean fetched -> " + bookService);
    }
}

```

## Expected Output

```
Context loaded, bean fetched -> com.library.service.BookService@6b95977
```

# Exercise 2: Implementing Dependency Injection

## 1. Updated applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository" />
    </bean>

```

```
</beans>
```

## 2. BookService.java (unchanged, setter already present)

BookService already has setBookRepository() from Exercise 1, so Spring can call it automatically because of the `<property>` tag above - no extra code needed on the Java side.

## 3. Test in LibraryManagementApplication.java

```
package com.library;

import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import com.library.service.BookService;

public class LibraryManagementApplication {
    public static void main(String[] args) {
        ApplicationContext context =
            new ClassPathXmlApplicationContext("applicationContext.xml");

        BookService bookService = (BookService) context.getBean("bookService");
        bookService.registerBook("The Great Gatsby");
    }
}
```

### Expected Output

```
Service layer received request for: The Great Gatsby
Book added to repository: The Great Gatsby
```

## Exercise 3: Implementing Logging with Spring AOP

This one adds an aspect that logs how long registerBook() takes to run, using @Around advice.

### 1. pom.xml - add AOP dependencies

```
<dependencies>
    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-context</artifactId>
        <version>${spring.version}</version>
    </dependency>

    <dependency>
        <groupId>org.springframework</groupId>
        <artifactId>spring-aop</artifactId>
        <version>${spring.version}</version>
    </dependency>

    <dependency>
        <groupId>org.aspectj</groupId>
        <artifactId>aspectjweaver</artifactId>
        <version>1.9.20</version>
    </dependency>
</dependencies>
```

### 2. LoggingAspect.java

*src/main/java/com/library/aspect/LoggingAspect.java*

```
package com.library.aspect;
```

```

import org.aspectj.lang.ProceedingJoinPoint;
import org.aspectj.lang.annotation.Around;
import org.aspectj.lang.annotation.Aspect;

@Aspect
public class LoggingAspect {

    @Around("execution(* com.library.service.*.*(..))")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {
        long start = System.currentTimeMillis();

        Object result = joinPoint.proceed();

        long end = System.currentTimeMillis();
        System.out.println(joinPoint.getSignature() + " executed in " + (end - start) + " ms");

        return result;
    }
}

```

### 3. applicationContext.xml - enable AspectJ support

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository" />
    </bean>

    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect" />

    <aop:aspectj-autoproxy />

</beans>

```

### Expected Output

```

Service layer received request for: The Great Gatsby
Book added to repository: The Great Gatsby
void com.library.service.BookService.registerBook(String) executed in 2 ms

```

## Exercise 4: Creating and Configuring a Maven Project

### Full pom.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0">
    <modelVersion>4.0.0</modelVersion>

    <groupId>com.library</groupId>
    <artifactId>LibraryManagement</artifactId>

```

```

<version>1.0-SNAPSHOT</version>
<packaging>war</packaging>

<properties>
  <maven.compiler.source>1.8</maven.compiler.source>
  <maven.compiler.target>1.8</maven.compiler.target>
  <spring.version>5.3.30</spring.version>
</properties>

<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-context</artifactId>
    <version>${spring.version}</version>
  </dependency>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-aop</artifactId>
    <version>${spring.version}</version>
  </dependency>

  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-webmvc</artifactId>
    <version>${spring.version}</version>
  </dependency>
</dependencies>

<build>
  <plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-compiler-plugin</artifactId>
      <version>3.11.0</version>
      <configuration>
        <source>1.8</source>
        <target>1.8</target>
      </configuration>
    </plugin>
  </plugins>
</build>
</project>

```

## Expected Output

Running mvn clean install from the project root should build without errors:

```

[INFO] Scanning for projects...
[INFO] Building LibraryManagement 1.0-SNAPSHOT
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----

```

## Exercise 5: Configuring the Spring IoC Container

### applicationContext.xml (central config)

```
<?xml version="1.0" encoding="UTF-8"?>
```

```

<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository" />
    </bean>

</beans>

```

## BookService.java - setter already present

No change needed, setBookRepository() from Exercise 1 is what the container calls when it reads the <property> tag.

## Expected Output

```

Service layer received request for: The Great Gatsby
Book added to repository: The Great Gatsby

```

# Exercise 6: Configuring Beans with Annotations

## 1. applicationContext.xml - enable component scan

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:context="http://www.springframework.org/schema/context"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/context
http://www.springframework.org/schema/context/spring-context.xsd">

    <context:component-scan base-package="com.library" />

</beans>

```

## 2. BookRepository.java

```

package com.library.repository;

import org.springframework.stereotype.Repository;

@Repository
public class BookRepository {

    public void addBook(String title) {
        System.out.println("Book added to repository: " + title);
    }

}

```

## 3. BookService.java

```

package com.library.service;

import org.springframework.beans.factory.annotation.Autowired;

```

```

import org.springframework.stereotype.Service;
import com.library.repository.BookRepository;

@Service
public class BookService {

    @Autowired
    private BookRepository bookRepository;

    public void registerBook(String title) {
        System.out.println("Service layer received request for: " + title);
        bookRepository.addBook(title);
    }
}

```

## Expected Output

```

Service layer received request for: The Great Gatsby
Book added to repository: The Great Gatsby

```

## Exercise 7: Implementing Constructor and Setter Injection

### 1. BookService.java

```

package com.library.service;

import com.library.repository.BookRepository;

public class BookService {

    private BookRepository bookRepository;

    // constructor injection
    public BookService(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    // setter injection
    public void setBookRepository(BookRepository bookRepository) {
        this.bookRepository = bookRepository;
    }

    public void registerBook(String title) {
        System.out.println("Service layer received request for: " + title);
        bookRepository.addBook(title);
    }
}

```

### 2. applicationContext.xml

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService">
        <constructor-arg ref="bookRepository" />
    </bean>
</beans>

```

```

        <property name="bookRepository" ref="bookRepository" />
    </bean>

</beans>

```

## Expected Output

```

Service layer received request for: The Great Gatsby
Book added to repository: The Great Gatsby

```

## Exercise 8: Implementing Basic AOP with Spring

### 1. LoggingAspect.java

```

package com.library.aspect;

public class LoggingAspect {

    public void logBefore() {
        System.out.println("[LOG] About to call registerBook()");
    }

    public void logAfter() {
        System.out.println("[LOG] Finished calling registerBook()");
    }
}

```

### 2. applicationContext.xml - register aspect with aop:config

```

<?xml version="1.0" encoding="UTF-8"?>
<beans xmlns="http://www.springframework.org/schema/beans"
       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
       xmlns:aop="http://www.springframework.org/schema/aop"
       xsi:schemaLocation="http://www.springframework.org/schema/beans
http://www.springframework.org/schema/beans/spring-beans.xsd
http://www.springframework.org/schema/aop
http://www.springframework.org/schema/aop/spring-aop.xsd">

    <bean id="bookRepository" class="com.library.repository.BookRepository" />

    <bean id="bookService" class="com.library.service.BookService">
        <property name="bookRepository" ref="bookRepository" />
    </bean>

    <bean id="loggingAspect" class="com.library.aspect.LoggingAspect" />

    <aop:config proxy-target-class="true">
        <aop:aspect ref="loggingAspect">
            <aop:pointcut id="registerBookPointcut"
                expression="execution(* com.library.service.BookService.registerBook(..))" />
            <aop:before method="logBefore" pointcut-ref="registerBookPointcut" />
            <aop:after method="logAfter" pointcut-ref="registerBookPointcut" />
        </aop:aspect>
    </aop:config>

</beans>

```

## Expected Output



```
[LOG] About to call registerBook()  
Service layer received request for: The Great Gatsby  
Book added to repository: The Great Gatsby  
[LOG] Finished calling registerBook()
```

## Exercise 9: Creating a Spring Boot Application

### 1. pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0">  
  <modelVersion>4.0.0</modelVersion>  
  
  <parent>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-parent</artifactId>  
    <version>2.7.18</version>  
  </parent>  
  
  <groupId>com.library</groupId>  
  <artifactId>LibraryManagement</artifactId>  
  <version>1.0-SNAPSHOT</version>  
  
  <properties>  
    <java.version>1.8</java.version>  
  </properties>  
  
  <dependencies>  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-web</artifactId>  
    </dependency>  
  
    <dependency>  
      <groupId>org.springframework.boot</groupId>  
      <artifactId>spring-boot-starter-data-jpa</artifactId>  
    </dependency>  
  
    <dependency>  
      <groupId>com.h2database</groupId>  
      <artifactId>h2</artifactId>  
      <scope>runtime</scope>  
    </dependency>  
  </dependencies>  
  
  <build>  
    <plugins>  
      <plugin>  
        <groupId>org.springframework.boot</groupId>  
        <artifactId>spring-boot-maven-plugin</artifactId>  
      </plugin>  
    </plugins>  
  </build>  
</project>
```

### 2. application.properties

**src/main/resources/application.properties**

```
spring.datasource.url=jdbc:h2:mem:librarydb  
spring.datasource.driver-class-name=org.h2.Driver
```

```
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
```

### 3. Book.java (entity)

```
package com.library.model;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;

@Entity
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String title;
    private String author;

    // getters and setters

    public Long getId() {
        return id;
    }

    public void setId(Long id) {
        this.id = id;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public String getAuthor() {
        return author;
    }

    public void setAuthor(String author) {
        this.author = author;
    }
}
```

### 4. BookRepository.java (JPA repository)

```
package com.library.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.library.model.Book;

public interface BookRepository extends JpaRepository<Book, Long> {
}
```

## 5. BookController.java

```
package com.library.controller;

import java.util.List;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;
import com.library.model.Book;
import com.library.repository.BookRepository;

@RestController
@RequestMapping("/api/books")
public class BookController {

    @Autowired
    private BookRepository bookRepository;

    @GetMapping
    public List<Book> getAllBooks() {
        return bookRepository.findAll();
    }

    @PostMapping
    public Book addBook(@RequestBody Book book) {
        return bookRepository.save(book);
    }
}
```

## 6. LibraryManagementApplication.java

```
package com.library;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LibraryManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(LibraryManagementApplication.class, args);
    }
}
```

## Expected Output

Console on startup:

```
Tomcat started on port(s): 8080 (http)
Started LibraryManagementApplication in 2.914 seconds
POST http://localhost:8080/api/books with body { "title": "The Great Gatsby", "author": "F. Scott Fitzgerald" }
returns:
```

```
{
  "id": 1,
  "title": "The Great Gatsby",
  "author": "F. Scott Fitzgerald"
}
```